Widzenie komputerowe

Detekcja obiektów, regresja położenia obiektu

Marcin Mikuła

Kod rozwiązania, prawie taki sam jak podany na zajęciach z drobnymi poprawkami umożliwiającymi uruchomienie.

```
# coding: utf-8

import os
import sys

import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

from PIL import Image, ImageDraw
from tensorflow.keras.layers import Input, Dense, Flatten, Conv2D,
MaxPool2D, BatchNormalization, Dropout #Import some useful libraries

print('Using TensorFlow version', tf.__version__) #Tensorflow version
check



emojis = {
    0: {'name': 'happy', 'file': '1F642.png'},
    1: {'name': 'laughing', 'file': '1F602.png'},
    2: {'name': 'skeptical', 'file': '1F928.png'},
    3: {'name': 'sad', 'file': '1F630.png'},
    4: {'name': 'cool', 'file': '1F60E.png'},
    5: {'name': 'whoa', 'file': '1F62F.png'},
    6: {'name': 'crying', 'file': '1F62D.png'},
    7: {'name': 'puking', 'file': '1F92E.png'},
    8: {'name': 'nervous', 'file': '1F62C.png'} #Some example images
}

plt.figure(figsize=(9, 9))

for i, (j, e) in enumerate(emojis.items()):
    plt.subplot(3, 3, i + 1)                                    #Plot
images 3x3
    plt.imshow(plt.imread(os.path.join('emojis', e['file']))) #imshow:
Show image, imread: Read image, os.path.join: Add file to the path
    plt.xlabel(e['name'])                                       #Add
class label on x axis
```

```
    plt.xticks([])
    plt.yticks([])
plt.show()


# ## Task 3: Create Examples

for class_id, values in emojis.items():
#Take class_id and values in emojis.items
    png_file = Image.open(os.path.join('emojis',
values['file'])).convert('RGBA')  #Open image in emojis, then convert
them to RGBA
    png_file.load()
#Load image
    new_file = Image.new("RGB", png_file.size, (255, 255, 255))
#Create a new image with all white
    new_file.paste(png_file, mask=png_file.split()[3])
#Paste emoji image on new image
    emojis[class_id]['image'] = new_file
#Add "image" key to emojis for keep PIL value


def create_example():
  class_id = np.random.randint(0, 9)
#Random choose an int from 0 to 9 for class_id
  image = np.ones((144, 144, 3)) * 255
#Create an 144x144x3 (white because of 255) array, call "image"
  row = np.random.randint(0, 72)
#Random choose an int from 0 to 72, call row
  col = np.random.randint(0, 72)
#Random choose an int from 0 to 72, call col
  image[row: row + 72, col: col + 72, :] =
np.array(emojis[class_id]['image'])#Add PIL value to "image" from
random row to row+72 and from random col to col+72
  return image.astype('uint8'), class_id, (row + 10) / 144, (col + 10)
/ 144  #Return image as uint8, class_id, row, col

#(original emoji image got 10 pixel margin on row and col, so we add 10
pix to get actual row and col, then we divide by 144 for normalize)



image, class_id, row, col = create_example()
```

```
plt.imshow(image);
plt.title('create_example()')
plt.show()


# ## Task 4: Plot Bounding Boxes

def plot_bounding_box(image, gt_coords, pred_coords=[], norm=False):
# image, ground truth coordinates, prediction coordinates,
normalization flag
  if norm:
# if normalization flag is True,(it means our image values are
normalized)
    image *= 255.
# then image values will be de-normalized
    image = image.astype('uint8')
# and it will become an unsigned integer as a type. So we can use the
image
  image = Image.fromarray(image)
# with "fromarray", we can display the image
  draw  = ImageDraw.Draw(image)
# for drawing the bounding box on the image

  row, col = gt_coords
  row *= 144
#multiply by 144 for de-normalized
  col *= 144
  draw.rectangle((col, row, col + 52, row + 52), outline='green',
width=3) #Draw a green rectangle on image with from row to row+52 and
from col to col+52(ground truth)

  if len(pred_coords) == 2:
#If length of prediction coordinates are 2(row and col)
    row, col = pred_coords
    row *= 144
    col *= 144
    draw.rectangle((col, row, col + 52, row + 52), outline='red',
width=3) #Draw a red rectangle on image with from row to row+52 and
from col to col+52(prediction)
  return image


print('plot_bounding_box')
```

```
image = plot_bounding_box(image, gt_coords=[row, col])
plt.imshow(image)
plt.title(emojis[class_id]['name'])
plt.show()


# ## Task 5: Data Generator

def data_generator(batch_size=16):
#batch_size: number of training examples
  while True:
    x_batch = np.zeros((batch_size, 144, 144, 3))
#Create a zeros array for images
    y_batch = np.zeros((batch_size, 9))
#Create a zeros array for classes
    bbox_batch = np.zeros((batch_size, 2))
#Create a zeros array for box(row, col)

    for i in range(0, batch_size):
      image, class_id, row, col = create_example()
      x_batch[i] = image / 255.
#image divide by 255 for normalizing
      y_batch[i, class_id] = 1.0
#Looks like => [0, 0, 0, 0, 1, 0, 0, 0, 0] (1 is for class_id if
class_id is 5)
      bbox_batch[i] = np.array([row, col])
    yield {'image': x_batch}, {'class_out': y_batch, 'box_out':
bbox_batch}#"yield is a keyword that is used like return, except the
function will return a generator"



print('data_generator()')
example, label = next(data_generator(1))           #Generate 1 example
image = example['image'][0]
class_id = np.argmax(label['class_out'][0])
coords = label['box_out'][0]

image = plot_bounding_box(image, coords, norm=True)
plt.imshow(image)
plt.title(emojis[class_id]['name'])
plt.show()
```

```
# ## Task 6: Model

input_ = Input(shape=(144, 144, 3), name='image')                #Input
layer, shape is the image shape

x = input_

for i in range(0, 5):
  n_filters = 2**(4 + i)
  x = Conv2D(n_filters, 3, activation='relu', padding='same')(x)
  x = BatchNormalization()(x)
  x = MaxPool2D(2)(x)

x = Flatten()(x)
x = Dense(256, activation='relu')(x)

class_out = Dense(9, activation='softmax', name='class_out')(x)# Dense
layer with 9 units for class
box_out   = Dense(2, name='box_out')(x)                          # Dense
layer with 2 units for bounding box(row, col)

model = tf.keras.models.Model(input_, [class_out, box_out])     # Build
model
model.summary()

with open('model_summary.txt', 'w') as sys.stdout:
  model.summary()
sys.stdout = sys.__stdout__


''' assign_add()
v = tf.Variable(1.)
v.assign(2.)            ---> 2.0
v.assign_add(0.5)       ---> 2.5
'''

# ## Task 7: Custom Metric: IoU

class IoU(tf.keras.metrics.Metric):
# Custom IoU class, inheritance of Metric class
  def __init__(self, **kwargs):
```

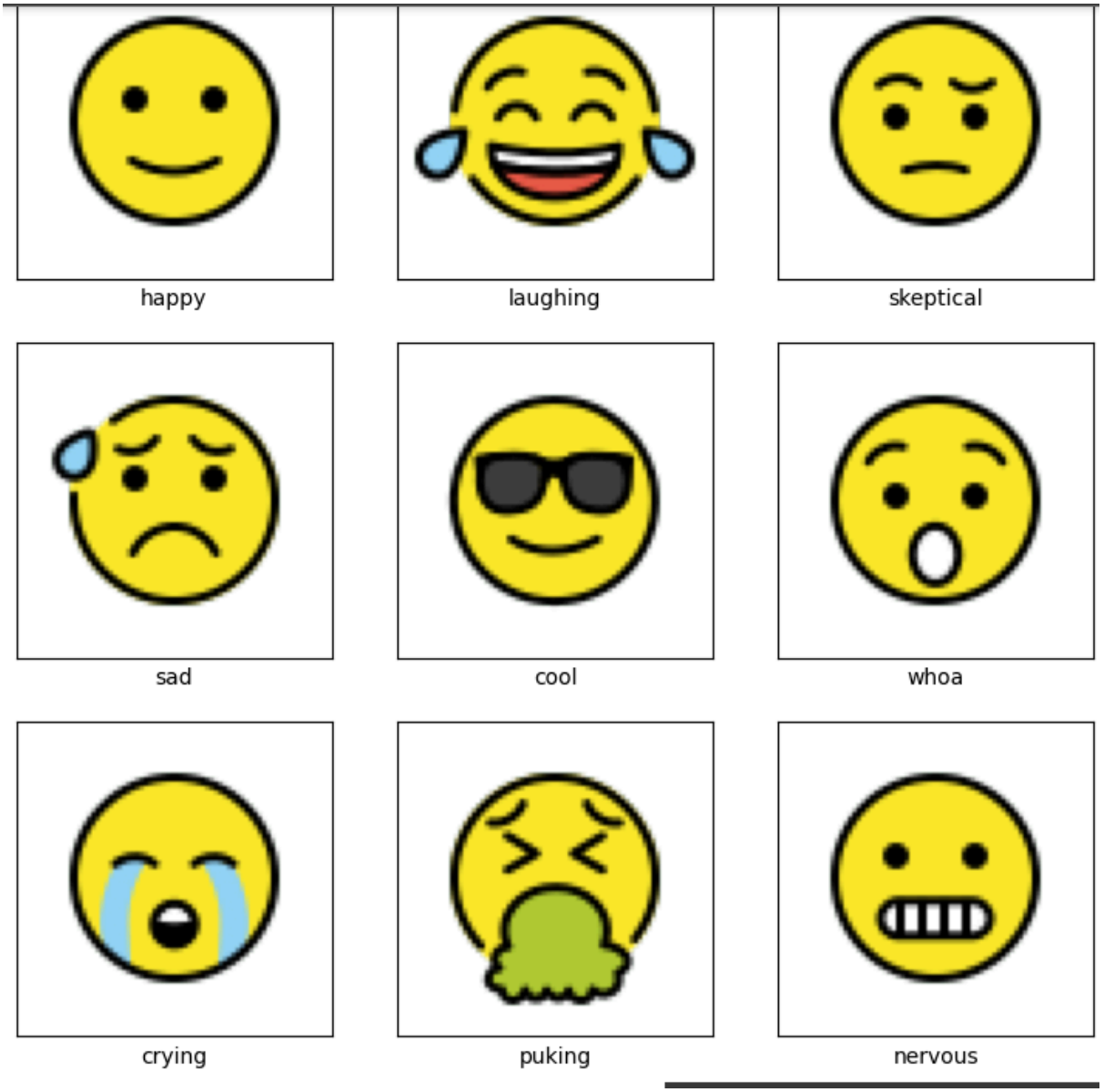
```
    super(IoU, self).__init__(**kwargs)
# super() allow us to use Metric class' arguments

    self.iou       = self.add_weight(name='iou', initializer='zeros')
# add_weight: "Adds a new variable to the layer"
    self.total_iou = self.add_weight(name='total_iou',
initializer='zeros')
    self.num_ex    = self.add_weight(name='num_ex',
initializer='zeros')

  def update_state(self, y_true, y_pred, sample_weight=None):
    def get_box(y):
# a function for getting bounding box coordinates
      rows, cols = y[:, 0], y[:, 1]
      rows, cols = rows * 144, cols * 144
      y1, y2 = rows, rows + 52
      x1, x2 = cols, cols + 52
      return x1, y1, x2, y2

    def get_area(x1, y1, x2, y2):
#a function for getting area of bounding box
      return tf.math.abs(x2 - x1) * tf.math.abs(y2 - y1)

    gt_x1, gt_y1, gt_x2, gt_y2 = get_box(y_true)
#getting ground truth bounding box coordinates
    p_x1, p_y1, p_x2, p_y2     = get_box(y_pred)
    gt_x1 = tf.cast(gt_x1, tf.float32)
#cast: "Casts a tensor to a new type"
    gt_y1 = tf.cast(gt_y1, tf.float32)
    gt_x2 = tf.cast(gt_x2, tf.float32)
    gt_y2 = tf.cast(gt_y2, tf.float32)

    i_x1 = tf.maximum(gt_x1, p_x1)
#return the maximum of gt_x1 and p_x1, assign to i_x1
    i_y1 = tf.maximum(gt_y1, p_y1)
#return the maximum of gt_x1 and p_x1, assign to i_x1
    i_x2 = tf.minimum(gt_x2, p_x2)
#return the maximum of gt_x1 and p_x1, assign to i_x1
    i_y2 = tf.minimum(gt_y2, p_y2)
#return the maximum of gt_x1 and p_x1, assign to i_x1

    i_area = get_area(i_x1, i_y1, i_x2, i_y2)
#area of intersection(or overlap)
```

```
    u_area = get_area(gt_x1, gt_y1, gt_x2, gt_y2) + get_area(p_x1,
p_y1, p_x2, p_y2) - i_area #area of union

    iou = tf.math.divide(i_area, u_area)
#calculate the iou
    self.num_ex.assign_add(1)
    self.total_iou.assign_add(tf.reduce_mean(iou))
#tf.reduce_mean():"Computes the mean of elements across dimensions of a
tensor"
    self.iou = tf.math.divide(self.total_iou, self.num_ex)
#total_iou divide by num_ex, then assign to iou

  def result(self):
    return self.iou

  def reset_state(self):
#reseting the state
    self.iou = self.add_weight(name='iou', initializer='zeros')
    self.total_iou = self.add_weight(name='total_iou',
initializer='zeros')
    self.num_ex = self.add_weight(name='num_ex', initializer='zeros')


# ## Task 8: Compile the Model

model.compile(
    loss={
        'class_out': 'categorical_crossentropy',            #we have 2
outputs, one of which is class_out and we use
"categorical_crossentropy" loss for it because class_out will be chosen
from an array of 9 classes
        'box_out': 'mse'                                    #one of
which is box_out, we use mse(mean squared error) loss for it because
box_out return 2 numeric value(row and col)
    },
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
    metrics={
        'class_out': 'accuracy',                            #we use
"accuracy" metric for class_out
        'box_out': IoU(name='iou')                          #and use
our custom IoU class for box_out metric
    }
)
```

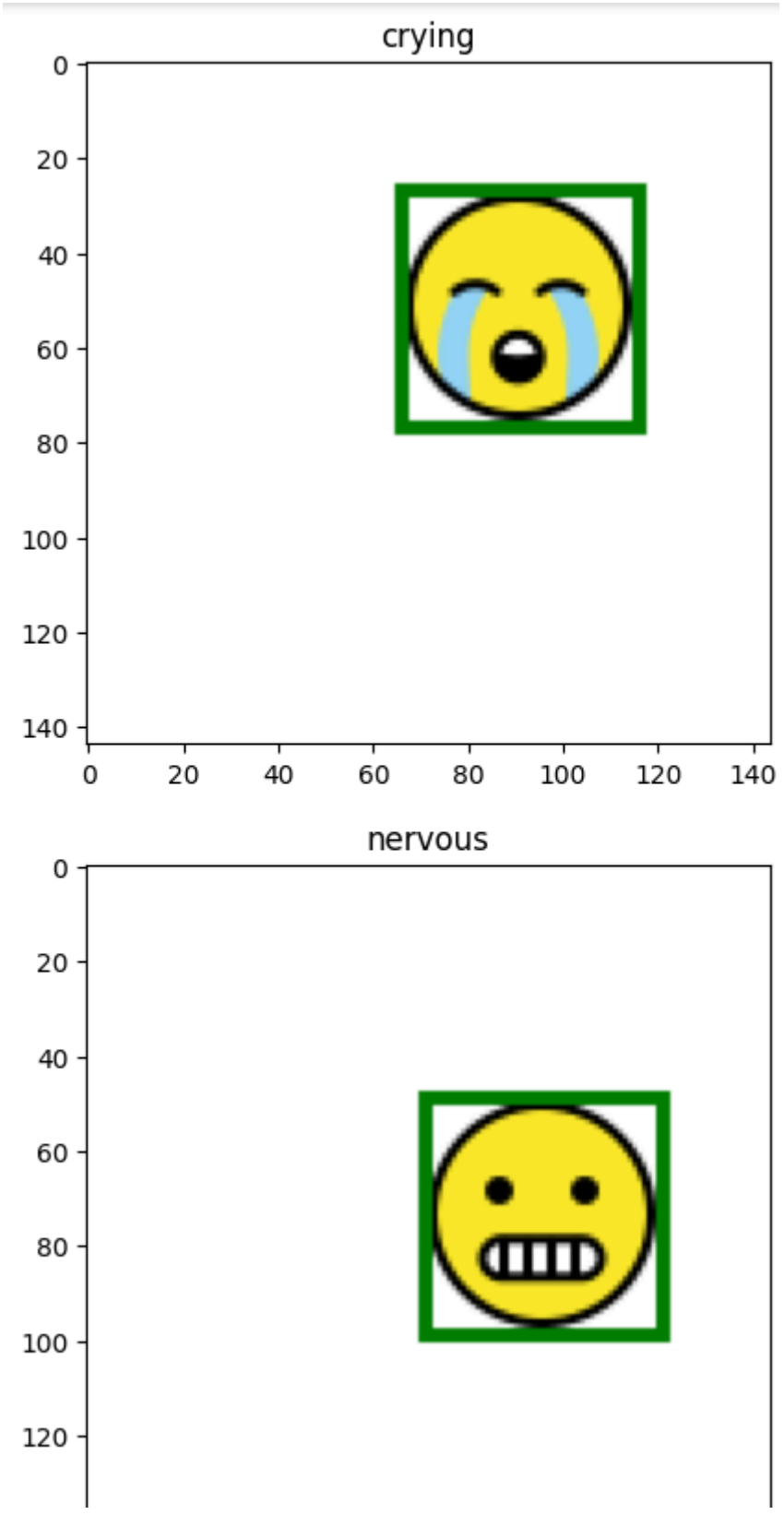
```
# ## Task 9: Custom Callback: Model Testing


def test_model(model, test_datagen):    #a function for test the model
  example, label = next(test_datagen)
  x = example['image']
  y = label['class_out']
  box = label['box_out']

  pred_y, pred_box = model.predict(x)

  pred_coords = pred_box[0]
  gt_coords = box[0]
  pred_class = np.argmax(pred_y[0])     #np.argmax: "Returns the
indices of the maximum values along an axis"
  image = x[0]

  gt = emojis[np.argmax(y[0])]['name']
  pred_class_name = emojis[pred_class]['name']

  image = plot_bounding_box(image, gt_coords, pred_coords, norm=True)
  color = 'green' if gt == pred_class_name else 'red'

  plt.imshow(image)
  plt.xlabel(f'Pred: {pred_class_name}', color=color)
  plt.ylabel(f'GT: {gt}', color=color)
  plt.xticks([])
  plt.yticks([])



def test(model):                    #a function for show the test result
  test_datagen = data_generator(1)

  plt.figure(figsize=(16, 4))
  for i in range(0, 6):
    plt.subplot(1, 6, i + 1)
    test_model( model, test_datagen)

  #plt.show()
```

```
print('model not trained - test(model)')
test(model)




class ShowTestImages(tf.keras.callbacks.Callback): #a custom callback
to show the results of the model at the end of each epoch
  def on_epoch_end(self, epoch, logs=None):
    test(self.model)


# ## Task 10: Model Training


def lr_schedule(epoch, lr):
  if (epoch + 1) % 5 == 0:
    lr *= 0.2              #at the end of every 5 epochs, the learning
rate will multiplied by 0.2 for gradient descent
  return max(lr, 3e-7)     #compare learning rate and 0.0000003, then
return the largest number. Because we want the minimum of learning rate
is 0.0000003


history = model.fit(
    data_generator(),
    #epochs=50,
    epochs=10,
    steps_per_epoch=500,
    callbacks=[
                ShowTestImages(),
#Custom callback
                tf.keras.callbacks.EarlyStopping(monitor='box_out_iou',
patience=3, mode='max'), #Monitoring the box_out_iou for 3 epochs and
if the quantity monitored has stopped increasing, then model.fit will
be stop
                tf.keras.callbacks.LearningRateScheduler(lr_schedule)
#"At the beginning of every epoch, this callback gets the updated
learning rate value from schedule(lr_schedule) function"
    ]
)
```

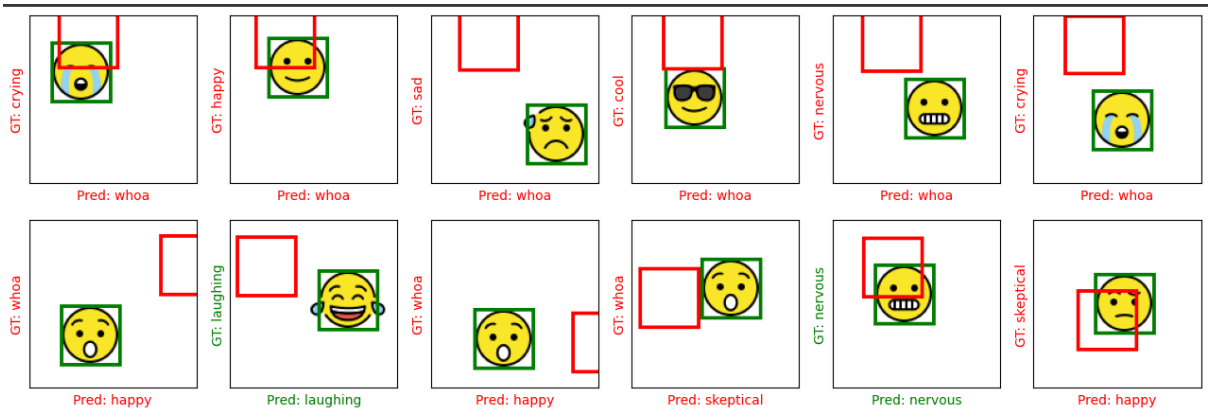
```
print(history.history.keys())    # dict_keys(['loss', 'class_out_loss',
'box_out_loss', 'class_out_accuracy', 'box_out_iou', 'lr'])


#plt.close()
plt.close("all")

test(model)
plt.show()
```



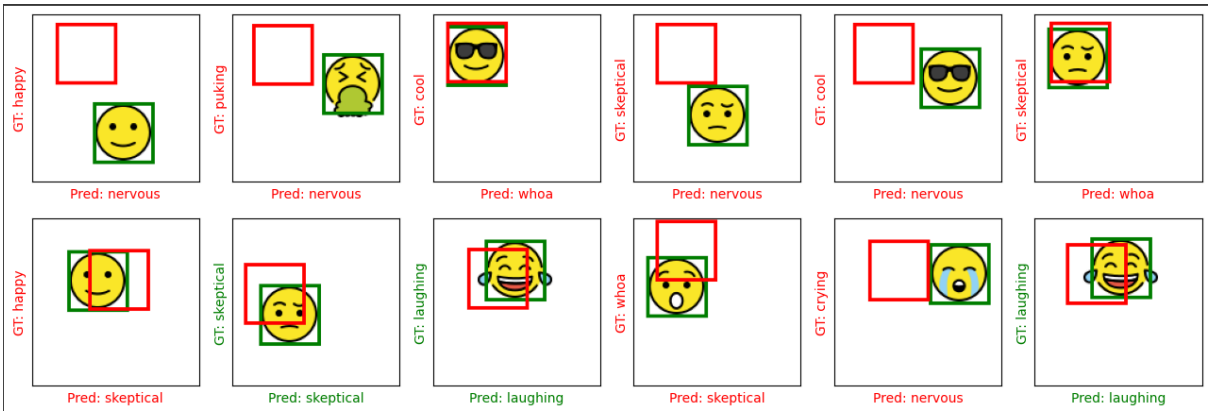
Obrazek 1. Input.


crying
0
20
40
60
80
100
120
140
0
20
40
60
80
100
120
140
nervous
0
20
40
60
80
100
120

Obrazek 2.



Obrazek 3. Wynik.



Obrazek 3. Wynik2.

Niestety kod zawiera błędy których nie udało mi się poprawić

```
TypeError                                 Traceback (most recent call last)
<ipython-input-6-2ab23faa26d9> in <cell line: 284>()
    282
    283
--> 284 history = model.fit(
    285     data_generator(),
    286     #epochs=50,
```

```
1 frames
/usr/local/lib/python3.10/dist-packages/tensorflow/python/eager/execute.py in
quick_execute(op_name, num_outputs, inputs, attrs, ctx, name)
     51   try:
     52     ctx.ensure_initialized()
---> 53     tensors = pywrap_tfe.TFE_Py_Execute(ctx._handle, device_name, op_name,
     54                                         inputs, attrs, num_outputs)
     55   except core._NotOkStatusException as e:
```

TypeError: <tf.Tensor 'truediv_1:0' shape=() dtype=float32> is out of scope and cannot be used here. Use return values, explicit Python locals or TensorFlow collections to access it.
Please see
https://www.tensorflow.org/guide/function#all_outputs_of_a_tffunction_must_be_return_values
for more information.